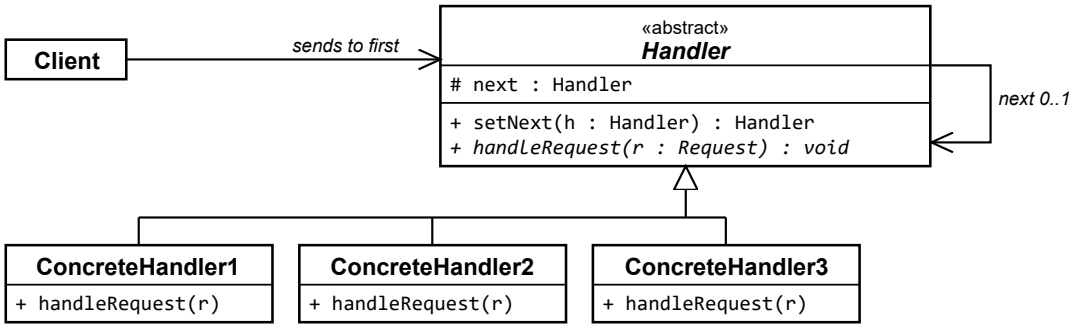
08 Chain of Responsibility (CoR)

Behavioral

Intent (GoF)	**Avoid coupling the sender of a request to its receiver** by giving more than one object a chance to handle the request. **Chain the receiving objects and pass the request along the chain until an object handles it.**
Use when	Several handlers could process a request and the **sender shouldn't decide which**; requests go through **levels / escalation / sequential checks**; handlers should be added, removed or reordered easily. Analogy: patient → doctor → specialist; try/catch.
Trigger words	"several levels of support / escalation", "approval limits (team leader, manager, director, CEO)", "sequence of checks that must occur in order and be easy to extend", "multiple sub-filters", "different priorities", "exception handling", "multiple handlers hard-coded in sequence".
Participants	**Handler** (abstract; `next` reference + `setNext()` + `handleRequest()`) · **ConcreteHandlers** (handle if they can, else forward) · **Request** (data: level/type/amount) · **Client** (builds the chain, sends to the first handler only).
Chain ends when	(1) a handler processes the request completely, or (2) the end of the chain is reached (request unhandled ⇒ add a fallback/default handler). Variant: **every** handler processes (validation/filter pipeline) and a failure stops the chain.

UML — generic



Self-association `next` on Handler = the chain link (aggregation to itself). Each concrete handler knows only its **next** handler. **Traditional approach** (`if type == PASSWORD → level1 … else if …`) makes the client know every handler, tightly coupled; adding a level changes the client.

Java — IT support escalation (lecture)

```
public class SupportRequest {
    private int level; private String description;
    public SupportRequest(int level, String d) { this.level =
level; description = d; }
    public int getLevel() { return level; }
    public String getDescription() { return description; }
}
public abstract class SupportHandler {
    protected SupportHandler nextHandler;
    public void setNextHandler(SupportHandler n) {
this.nextHandler = n; }
    public abstract void handleRequest(SupportRequest r);
}
public class Level1Support extends SupportHandler {
    public void handleRequest(SupportRequest r) {
        if (r.getLevel() == 1)
            System.out.println("Level 1 handled: " +
r.getDescription());
        else if (nextHandler != null)
            nextHandler.handleRequest(r);       // pass along
    }
}
// Level2Support (level==2), Level3Support (level==3): same
shape
public class Administrator extends SupportHandler {   //
fallback
    public void handleRequest(SupportRequest r) {
        System.out.println("Admin handled: " +
r.getDescription());
    }
}
```

```
// CLIENT - build chain, send to FIRST only
SupportHandler l1 = new Level1Support();
SupportHandler l2 = new Level2Support();
SupportHandler l3 = new Level3Support();
SupportHandler admin = new Administrator();
l1.setNextHandler(l2);
l2.setNextHandler(l3);
l3.setNextHandler(admin);

l1.handleRequest(new SupportRequest(3, "Network is
unavailable"));
// L1: level 3? no -> L2: no -> L3: yes
// Output: Level 3 handled: Network is unavailable
```

Trace: request level 2	Result
Level1 (level==1?)	no → forward
Level2 (level==2?)	**yes → handled, stop**
Level3, Admin	not reached

Expense approval (in-class)	Limit
TeamLeaderApprover	≤ $1,000
ManagerApprover	$1,001 – $5,000
DirectorApprover	$5,001 – $20,000
CEOApprover	> $20,000 (any)

$750 → TL; $3,500 → Manager; $12,000 → Director; $50,000 → CEO.

Advantages / challenges → mechanism

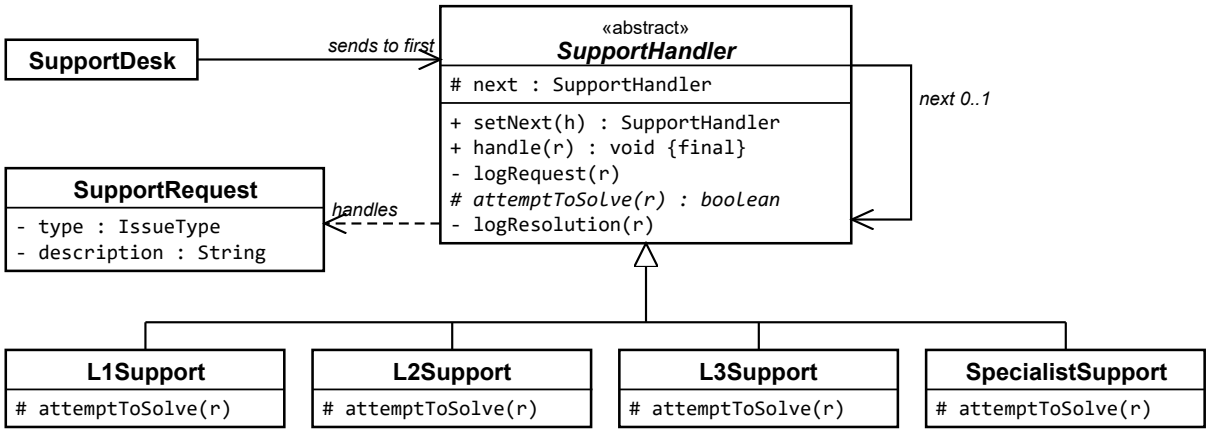
Advantages (slides)	Challenges (slides)	Mechanism to overcome
More than one object can handle a request	**No guarantee** the request is handled (end of chain)	Add a **default/fallback handler** at the end (Administrator, Specialist, “manual review”)
Nodes added/removed dynamically; order can be shuffled	**Debugging is tricky**	Log at every handler (who received, who handled); unit-test each handler
A handler doesn’t need to know how the next works (SRP)	**Unnecessary handlers** ⇒ **inefficiency** / long chains	**Set priorities** (performance, accuracy, less complex first); **bypass** handlers based on request info; keep chain short
Decouples sender and receivers; can improve HA/DR (backup handlers)	Order sensitive; cycles if mis-linked	Build the chain in one place (builder/config); validate no cycles

Priorities / ordering criteria (slides): performance (fast handlers first), accuracy, complexity (simple first), cost, availability, severity (high-priority checks first), business priority. “Should the fastest handler always be first?” — usually cheap, high-rejection checks first so expensive ones run less; but severity or accuracy may override. **CoR vs State:** CoR = “*who* will handle this request?” (request moves along handlers); State = “*how* should I behave now?” (context delegates to its current state).

Questions & model answers

Technical support: L1 (password reset, blocked accounts), L2 (installation errors), L3 (DB connectivity, performance), Specialist (rare problems). Explain CoR (identify request, handler, concrete handlers) (4); UML (6); Java for a DB-connectivity request (6); enforce log → attempt → log for every level (4). *Quiz 7*

Request = `SupportRequest` (issue type + description). **Handler** = abstract `SupportHandler` holding `next`. **Concrete handlers** = `L1Support`, `L2Support`, `L3Support`, `SpecialistSupport` (end of chain, handles everything left). **Client** = support desk; it sends every request to L1 and never decides who solves it. Each level solves what it can and escalates the rest; levels can be added/reordered without touching the client. The fixed procedure is enforced by a **Template Method** inside the handler (`final handle()`).

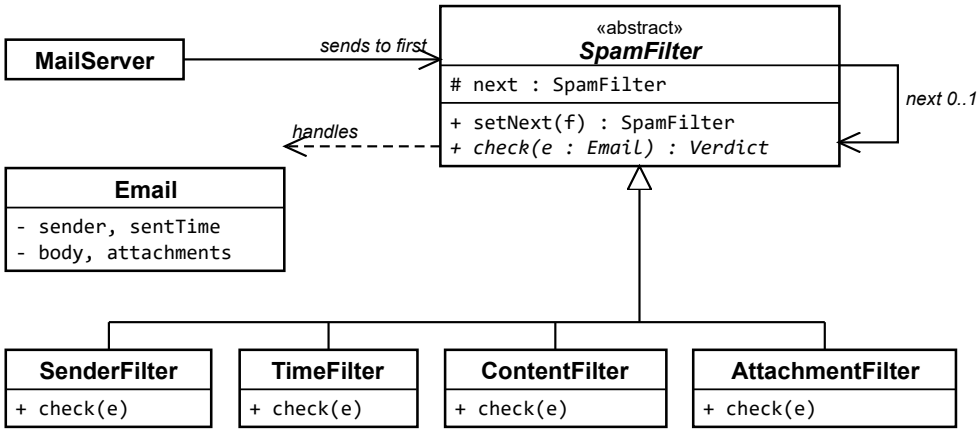



```
public enum IssueType { PASSWORD_RESET, ACCOUNT_BLOCKED, INSTALLATION_ERROR, DB_CONNECTIVITY, PERFORMANCE, OTHER }
public class SupportRequest {
    private final IssueType type; private final String description;
    public SupportRequest(IssueType t, String d) { type = t; description = d; }
    public IssueType getType() { return type; }  public String getDescription() { return description; }
}
public abstract class SupportHandler {
    protected SupportHandler next;
    public SupportHandler setNext(SupportHandler n) { this.next = n; return n; }   // returns n to allow chaining

    public final void handle(SupportRequest r) {              // TEMPLATE METHOD (Q4)
        logRequest(r);
        boolean solved = attemptToSolve(r);                    // level-specific logic
        logResolution(r, solved);
        if (!solved && next != null) next.handle(r);           // CoR: escalate
    }
    private void logRequest(SupportRequest r) { System.out.println(getClass().getSimpleName() + " received " +
r.getType()); }
    private void logResolution(SupportRequest r, boolean ok) { System.out.println(getClass().getSimpleName() + (ok ? "
RESOLVED" : " escalated")); }
    protected abstract boolean attemptToSolve(SupportRequest r);
}
public class L1Support extends SupportHandler {
    protected boolean attemptToSolve(SupportRequest r) {
        return r.getType() == IssueType.PASSWORD_RESET || r.getType() == IssueType.ACCOUNT_BLOCKED;
    }
}
public class L2Support extends SupportHandler {
    protected boolean attemptToSolve(SupportRequest r) { return r.getType() == IssueType.INSTALLATION_ERROR; }
```

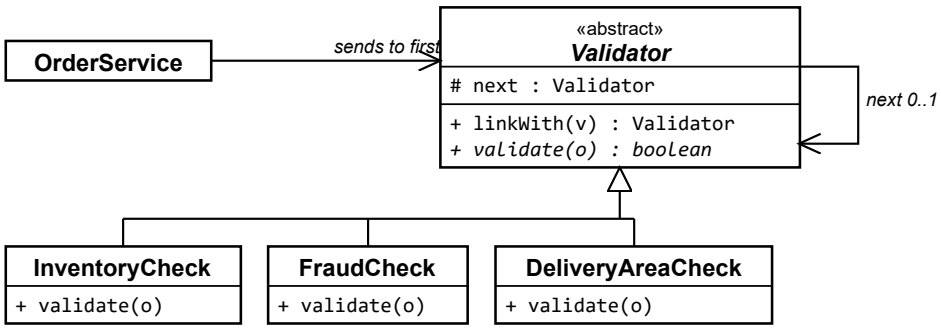
```
}
public class L3Support extends SupportHandler {
    protected boolean attemptToSolve(SupportRequest r) {
        if (r.getType() == IssueType.DB_CONNECTIVITY || r.getType() == IssueType.PERFORMANCE) {
            System.out.println("L3: checking DB server, connection pool, restarting service"); return true;
        }
        return false;
    }
}
public class SpecialistSupport extends SupportHandler {
    protected boolean attemptToSolve(SupportRequest r) { System.out.println("Specialist investigating"); return true; }
}
// CLIENT
SupportHandler chain = new L1Support();
chain.setNext(new L2Support()).setNext(new L3Support()).setNext(new SpecialistSupport());
chain.handle(new SupportRequest(IssueType.DB_CONNECTIVITY, "App cannot connect to MySQL"));
/* L1Support received DB_CONNECTIVITY / L1Support escalated
   L2Support received DB_CONNECTIVITY / L2Support escalated
   L3Support received DB_CONNECTIVITY / L3: checking DB ... / L3Support RESOLVED   (Specialist not reached) */
```

Email spam filter with sub-filters (sender, sent time, content, attachment): diagram, pattern & reasons, prioritisation criteria, adding sub-components, HA & DR. *2022 Q3 (25)*



CoR: each sub-filter is a handler; the email passes Sender → Time → Content → Attachment; any filter can mark SPAM and stop, otherwise forward; end of chain = deliver to inbox. Reasons: decoupled, each filter one responsibility, filters added/reordered without changing the mail server. **Prioritise** by: cost/performance (cheap sender blacklist and time checks first, expensive content NLP and attachment virus scan last), rejection rate (filters that catch most spam early), accuracy/false positives, severity (malware attachment may go first for security). **Adding sub-components:** new class extending `SpamFilter` (e.g. `LinkReputationFilter`), insert with `setNext` or from a config list — no change to existing filters (OCP). **HA & DR:** stateless filter chain replicated on several servers behind a load balancer; a backup handler/replica if one filter service fails (skip-on-timeout or fallback filter); message queue so mails aren't lost; rules and blacklists replicated to a second region with backups; failover to DR site.

Order checks: inventory → fraud → delivery-area; maintainable & extendable. / Library: membership → availability → fines, in order, easy to extend. *2025 Q4.1(b), 3.4; Q3.1(a), 3.4 (10)*



```
public abstract class Validator {
    private Validator next;
    public Validator linkWith(Validator n) { this.next = n; return n; }
    public boolean validate(Order o) {
        if (!check(o)) return false;                // fail -> stop chain
        return next == null || next.validate(o);    // pass -> next check
    }
    protected abstract boolean check(Order o);
}
public class InventoryCheck extends Validator { protected boolean check(Order o) { return Stock.has(o.getItems()); } }
public class FraudCheck extends Validator { protected boolean check(Order o) { return FraudService.score(o) < 0.7; } }
public class DeliveryAreaCheck extends Validator { protected boolean check(Order o) { return Zones.covers(o.getAddress());
} }

Validator v = new InventoryCheck();  v.linkWith(new FraudCheck()).linkWith(new DeliveryAreaCheck());
if (v.validate(order)) paymentProcessor.handlePayment(order);
```

Library: `MembershipCheck` → `AvailabilityCheck` → `FineCheck` in the same way; a failed check stops and returns the reason ("membership expired"). New check (e.g. `BorrowLimitCheck`) = new class + one link.

Exception-handling system that handles different priorities. / Include exception handling for different types. / Different levels of authorization. *2023 Q4.3; 2024 Q4.3, 4.5*

CoR: `ExceptionHandler` chain ordered by priority/severity — `ValidationErrorHandler` (low: show message) → `PaymentErrorHandler` (retry / alternate method) → `SystemErrorHandler` (log, alert admin) → `DefaultHandler` (generic error page). Like try/catch blocks: if one can't handle, control passes on. Authorization levels: `Authenticated` → `RoleCheck` → `PermissionCheck` → `ApprovalLimit` handlers, each higher level handling what lower ones can't (same as expense approval).

Mistakes: client calling every handler itself (that's the if-else approach) · forgetting `next != null` check · no fallback handler · drawing association from concrete handlers to each other instead of the `next` self-association on Handler · confusing with State.